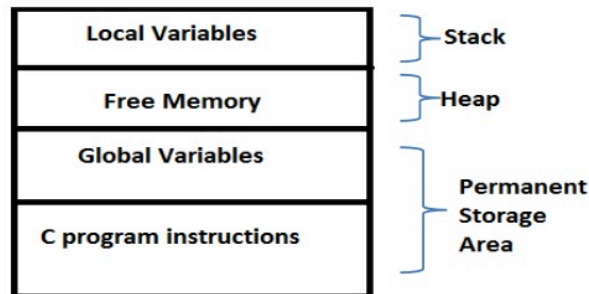


1. Dynamic Memory Allocation

1.1 Meaning of Dynamic Memory Allocation

- **Dynamic Memory Allocation** means allocating memory **during program execution (run time)**.
- Memory size can be **changed as per requirement**.
- Memory is allocated from **Heap memory**.
- Useful when we **do not know the required memory size in advance**.

Memory Allocation Process



Static vs Dynamic Memory

Static Memory	Dynamic Memory
Size fixed at compile time	Size decided at run time
Less flexible	More flexible
Example: <code>int a[10];</code>	Example: <code>malloc()</code>

To use these, you must include `<stdlib.h>`.

- 1) **malloc** : It allocates requested size of bytes and returns a pointer to the first byte of the allocated space.
- 2) **calloc** : Allocates space for an array of elements ,initializes them and then returns a pointer to the memory.
- 3) **free** : Frees previously allocated space.
- 4) **realloc** : Modifies the size of previously allocated space.

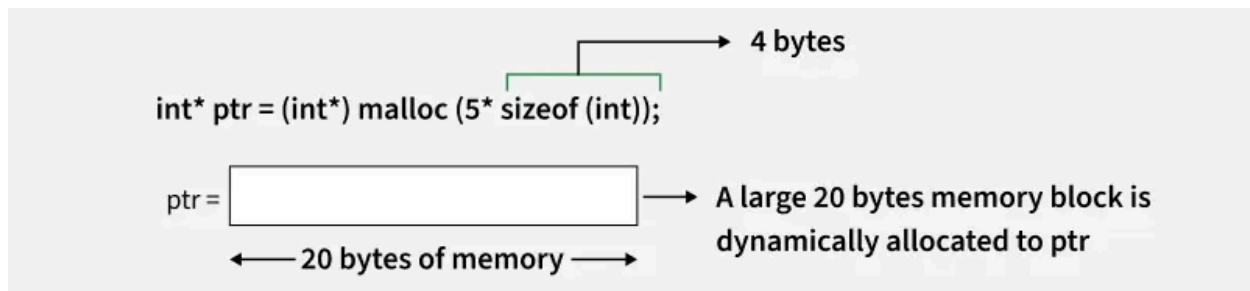
1.2 Dynamic Memory Allocation Functions

These functions are available in `<stdlib.h>` header file.

(a) malloc() – Memory Allocation

- Allocates a single **block** of memory.
- It allocates requested size of bytes and returns a pointer to the first byte of the allocated space.
- Memory contains **garbage values**.
- Syntax:

ptr = (datatype*) malloc(size);



Example 1:

Suppose you want to store marks of students but you don't know how many students there are.

So you ask the user.

Then you create memory dynamically.

```
int n;  
scanf("%d",&n);  
  
int *p;  
p = (int*) malloc(n * sizeof(int));
```

Example2:

```
#include <stdio.h>  
#include <stdlib.h>  
  
int main()
```

```

{
    int *p;
    p = (int*) malloc(5 * sizeof(int));

    if(p == NULL)
        printf("Memory not allocated");
    else
        printf("Memory allocated successfully");

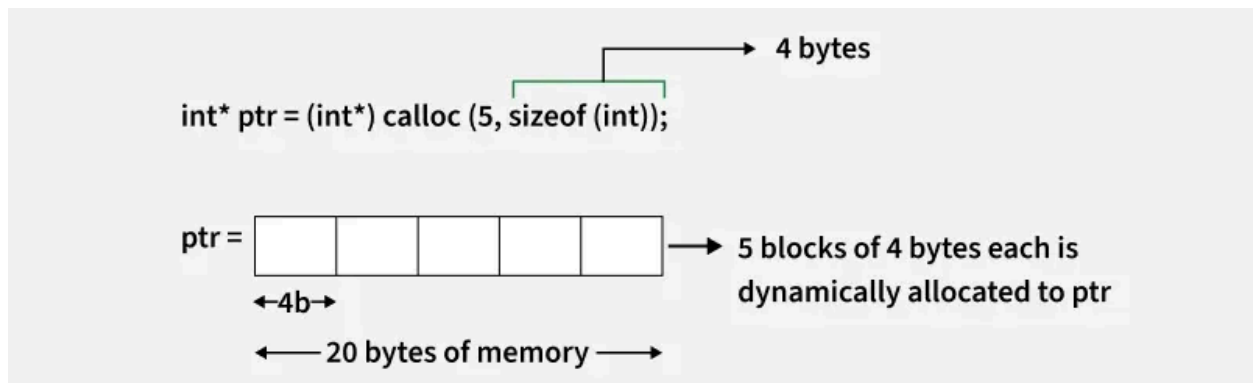
    return 0;
}

```

(b) calloc() – Continuous Allocation

- Allocates **multiple blocks** of memory.
- Allocates space for an array of elements ,initializes them and then returns a pointer to the memory.
- Initializes memory with **zero (0)**.
- Syntax:

ptr = (datatype*) calloc(n, size);



Example:

```

#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p;

```

```
p = (int*) calloc(5, sizeof(int));

printf("Memory allocated using calloc");
return 0;
}
```

(c) realloc() – Reallocation

- Changes the size of previously allocated memory (makes it bigger or smaller) without losing the data inside.
- Syntax:

```
ptr = realloc(ptr, new_size);
```

Example:

```
p = realloc(p, 10 * sizeof(int));
```

(d) free() – Free Memory

- Releases the memory back to the system. If you don't do this, you get a "Memory Leak," which slows down the computer.
- Prevents **memory waste**.
- Syntax:

```
free(ptr);
```

2. Pre-processor in C

2.1 Introduction of Pre-processor

The **C Preprocessor** is a program that processes the source code **before the actual compilation starts**.

It handles special commands called **preprocessor directives**, which begin with the symbol **#**.

These directives instruct the compiler to perform certain tasks such as:

- Including header files
- Defining macros

- Conditional compilation

Key Characteristics

1. Preprocessor directives begin with `#`.
2. They **do not end with a semicolon (;)**.
3. They are executed **before compilation**.
4. They **do not generate machine code directly**.

Directive	Purpose
<code>#include</code>	Includes header files
<code>#define</code>	Defines macros
<code>#undef</code>	Undefines macros
<code>#ifdef</code>	Conditional compilation
<code>#ifndef</code>	Conditional compilation
<code>#if</code>	Compile code conditionally

2.2 Macro Substitution

- This replaces a name with a value or a snippet of code throughout the program.
- Created using `#define`.

Example:

```
#define PI 3.14
#include <stdio.h>
int main()
{
    printf("Value of PI = %f", PI);
    return 0;
}
// Wherever you write PI, the pre-processor replaces it with 3.14
```

2.3 File Inclusion

File inclusion in C means inserting the contents of one file into another file during compilation. It is mainly used to include **header files** that contain function declarations, macros, constants, and definitions.

File inclusion is done using the `#include` **preprocessor directive**.

The C **preprocessor** processes these directives **before the actual compilation** begins.

Syntax of File Inclusion

```
#include <filename>
#include "filename"
```

There are **two types of file inclusion** in C.

1. System File Inclusion

Explanation

- Used to include **standard header files** provided by the C library.

- The compiler searches for these files in **standard system directories**.

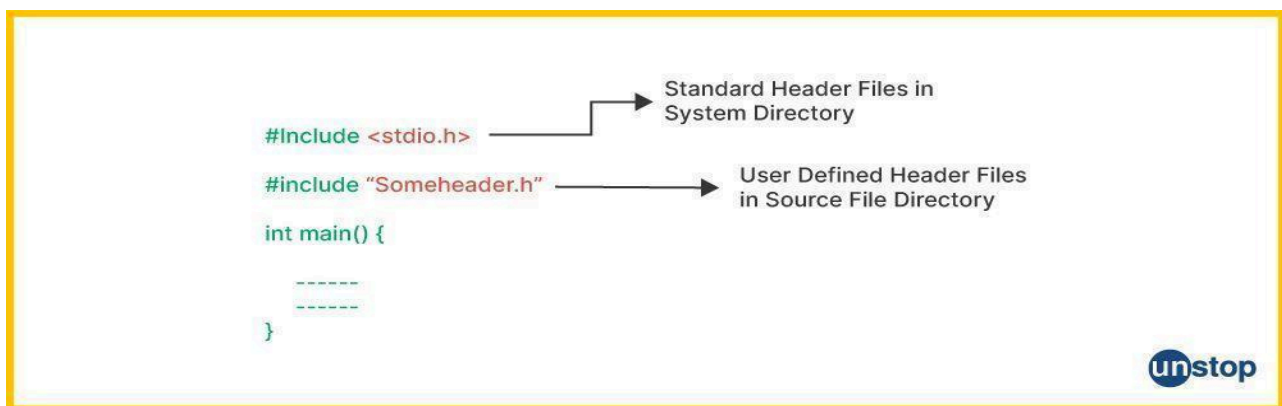
Example

```
#include <stdio.h>
int main()
{
    printf("Hello World");
    return 0;
}
```

Common Standard Header Files

Header File	Purpose
<code>stdio.h</code>	Input and Output functions
<code>conio.h</code>	Console input/output (Turbo C)
<code>math.h</code>	Mathematical functions
<code>string.h</code>	String manipulation
<code>stdlib.h</code>	Memory allocation, conversion

2. User Defined File Inclusion



Syntax

```
#include "filename"
```

Explanation

- Used to include **user-created header files**.
- The compiler first searches the **current directory** and then system directories.

Example

Header file: **myfile.h**

```
#define PI 3.14
```

Main Program

```
#include <stdio.h>
```

```
#include "myfile.h"
```

```
int main()
{
    printf("Value of PI = %f", PI);
    return 0;
}
```

Output

Value of PI = 3.140000

Advantages of File Inclusion

1. **Code Reusability** – Same file can be used in multiple programs.
2. **Easy Maintenance** – Changes in header file automatically apply everywhere.
3. **Modular Programming** – Large programs can be divided into smaller files.
4. **Reduces Code Duplication**

Disadvantages of File Inclusion

1. **Compilation Time Increases** if many files are included.
2. **Multiple Inclusion Problem** may occur.

Concept	Description
Preprocessor	Processes code before compilation
Macro Substitution	Replaces macro names with values
File Inclusion	Inserts content of another file

2.4 Compiler Control Directives (Conditional Compilation)

These tell the compiler to skip or include certain parts of the code based on conditions. This is useful for making code work on different operating systems

Directive	Use
#if	Condition true
#else	Executes when condition is false
#elif	Else-if condition
#endif	End condition

Example:

```
#define X 10
```

```
#if X > 5
    printf("X is greater than 5");
#else
    printf("X is smaller");
#endif
```

3. Program Development Steps

3.1 Program Design

- Understand the problem.
- Decide **input, process, output**.
- Use **algorithms or flowchart**.

Stages of Program Design:

1. **Problem Analysis** – Understand the problem clearly.
2. **Outlining Program Structure** – Decide main steps of the program.
3. **Algorithm Development** – Write step-by-step solution.
4. **Selection of Control Structure** – Choose if, loop, switch, etc.

3.2 Program Coding

- Write code using correct **syntax**.
- Follow proper **indentation**.
- Use meaningful **variable names**.

Elements of Good Coding Style:

1. **Internal Documentation** – Use comments.
2. **Construction of Statements** – Write correct statements.
3. **Generality of Program** – Should work for different inputs.
4. **Input / Output Format** – Clear input and output display.

4. Common Programming Errors

- Missing or misuse of semicolon ;
- Missing braces { }
- Missing or wrong quotes " "
- Wrong comment symbols
- Undeclared variables
- Wrong operator precedence
- Wrong use of increment/decrement operators
- Function not declared or parameters missing

- Data type mismatch in function calls
- Missing `&` in `scanf()`
- Array index out of range
- No space for null character `\0` in string
- Using uninitialized pointers
- Missing `*` or `&` operators
- Missing parentheses in pointer expressions
- Missing parentheses in macros

4.1 Syntax Errors

- Mistakes in grammar.
- Detected by compiler.

`printf("Hello") // missing semicolon`

4.2 Logical Errors

- Program runs but gives **wrong output**.

`sum = a - b; // should be a + b`

4.3 Runtime Errors

- Occur while program is running.
- **Example: division by zero.**

5. Program Testing and Debugging

Program Testing

- Checking whether program gives **correct output**.
- Use **different test values**.

Debugging

- Finding and correcting errors.
- Use:
 - Print statements
 - Step-by-step execution

6. Program Efficiency

Meaning

- Efficient program uses **less time and less memory**.

Tips for Efficiency

- Avoid unnecessary loops
- Free unused memory
- Use proper data types
- Write simple logic